

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

CHƯƠNG 7. TÀI LIỆU THIẾT KẾ TỐT

(Good Design Specification)

HỆ THỐNG QUẢN LÝ TRUNG TÂM CHĂM SÓC THÚ CÙNG
PET SERVICE

Học phần: Phát triển phần mềm theo chuẩn kỹ năng ITSS

GVHD: ThS. Nguyễn Mạnh Tuấn

NHÓM 23

STT	Họ và tên sinh viên
1	Trần Đức Nam Anh
2	Nguyễn Đức Hiếu
3	Triệu Trường Giang

Hà Nội, tháng 6 năm 2026

MỤC LỤC

7.1. Thiết kế mô-đun (Modularity)	3
7.1.1. Khái niệm và vai trò	3
7.1.2. Đặc điểm của một thiết kế mô-đun tốt	3
7.2. Coupling và Cohesion	4
7.2.1. Các mức Coupling	4
7.2.2. Các mức Cohesion	4
7.3. Tổng quan các nguyên lý SOLID	4
7.4. Áp dụng Good Design vào dự án Pet Service	5
7.4.1. Thiết kế mô-đun của hệ thống	5
7.4.1.1. Kiến trúc tổng thể	5
7.4.1.2. Modularity phía Backend	6
7.4.1.3. Modularity phía Frontend	7
7.4.2. Đánh giá Coupling và Cohesion	8
7.4.3. Đánh giá các nguyên lý SOLID	10
7.5. Đánh giá theo các tính chất mô-đun lý tưởng	12
7.6. Các điểm cần tiếp tục hoàn thiện	13
7.7. Lộ trình cải thiện thiết kế	14
7.8. Kết luận	15

7.1. Thiết kế mô-đun (Modularity)

7.1.1. Khái niệm và vai trò

Mô-đun là một đơn vị tương đối độc lập trong hệ thống phần mềm, có chức năng, dữ liệu đầu vào, dữ liệu đầu ra và phạm vi trách nhiệm rõ ràng. Mô-đun có thể là một component, một service, một route group, một tập hợp type hoặc một thư mục nghiệp vụ. Thiết kế mô-đun không đơn giản là chia code thành nhiều file; điều quan trọng hơn là mỗi phần được chia ra phải có ý nghĩa và có ranh giới giao tiếp rõ.

Khi hệ thống được chia mô-đun hợp lý, thành viên trong nhóm có thể đọc, sửa và kiểm thử một phần mà không cần hiểu toàn bộ ứng dụng. Điều này đặc biệt cần thiết với Pet Service vì dự án có nhiều tác nhân cùng sử dụng: khách hàng, nhân viên, bác sĩ và quản trị viên. Mỗi vai trò có màn hình, dữ liệu và quy trình nghiệp vụ riêng nhưng vẫn phải phối hợp với nhau trong các luồng đặt lịch, khám bệnh, grooming, lưu trú và thanh toán.

Lợi ích chính của tính mô-đun:

- Dễ hiểu: cấu trúc thư mục thể hiện được các vùng chức năng của hệ thống.
- Dễ phân công: thành viên có thể làm việc trên các module khác nhau và giảm xung đột mã nguồn.
- Dễ kiểm thử: lỗi được khoanh vùng trong một nhóm chức năng nhỏ hơn.
- Dễ bảo trì: thay đổi trong một nghiệp vụ ít làm ảnh hưởng sang nghiệp vụ khác.
- Dễ tái sử dụng: component UI, helper và service có thể dùng lại ở nhiều màn hình.

7.1.2. Đặc điểm của một thiết kế mô-đun tốt

Tính chất	Ý nghĩa trong thiết kế phần mềm
Decomposable	Có thể phân rã hệ thống thành các phần nhỏ để giảm độ phức tạp.
Composable	Các mô-đun có thể lắp ghép và phối hợp thành một luồng hoàn chỉnh.
Understandable	Có thể hiểu và kiểm thử một mô-đun mà không phải đọc quá nhiều phần khác.
Continuity	Một thay đổi nhỏ trong yêu cầu chỉ kéo theo thay đổi ở số ít mô-đun.
Isolation	Lỗi hoặc thay đổi hạ tầng được giới hạn, không lan truyền toàn hệ thống.

Nhận xét: Thiết kế tốt là sự cân bằng. Chia quá ít sẽ tạo file lớn và khó bảo trì; chia quá nhỏ lại làm tăng số lượng phụ thuộc và khiến luồng xử lý bị phân mảnh.

7.2. Coupling và Cohesion

Mục tiêu quan trọng của thiết kế phần mềm là đạt High Cohesion và Low Coupling: các thành phần trong cùng một mô-đun liên quan chặt chẽ với nhau, trong khi sự phụ thuộc giữa các mô-đun được giữ ở mức cần thiết và thông qua contract rõ ràng.

7.2.1. Các mức Coupling

Mức độ	Mô tả	Đánh giá
Content	Một mô-đun can thiệp trực tiếp vào chi tiết nội bộ của mô-đun khác.	Xấu nhất
Common	Nhiều mô-đun dùng chung trạng thái hoặc dữ liệu toàn cục.	Rất kém
Control	Truyền cờ điều khiển để quyết định luồng xử lý bên trong mô-đun khác.	Kém
Stamp	Truyền cả cấu trúc lớn nhưng bên nhận chỉ dùng một phần nhỏ.	Trung bình
Data	Chỉ truyền dữ liệu cần thiết thông qua tham số hoặc DTO rõ ràng.	Tốt nhất

7.2.2. Các mức Cohesion

Mức độ	Mô tả	Đánh giá
Coincidental	Các phần tử được gom ngẫu nhiên, gần như không liên quan.	Thấp
Logical	Nhóm theo một ý nghĩa chung nhưng nhiệm vụ cụ thể khác nhau.	Rất kém
Temporal	Nhóm vì được thực hiện cùng thời điểm.	Kém
Procedural	Nhóm vì được gọi theo một trình tự xử lý.	Trung bình
Communicational	Các phần cùng thao tác trên một tập dữ liệu chung.	Khá
Functional	Mọi phần phối hợp để hoàn thành một trách nhiệm rõ ràng.	Tốt nhất

7.3. Tổng quan các nguyên lý SOLID

SOLID gồm SRP, OCP, LSP, ISP và DIP. Các nguyên lý này không chỉ dành cho class hướng đối tượng mà còn có thể áp dụng cho component React, module JavaScript, service, middleware và contract TypeScript.

7.4. Áp dụng Good Design vào dự án Pet Service

Pet Service là ứng dụng fullstack quản lý trung tâm chăm sóc thú cưng. Frontend sử dụng React 18, Vite và TypeScript; backend sử dụng Express, Prisma/PostgreSQL và Supabase server client. Hệ thống hỗ trợ bốn vai trò chính: customer, staff, doctor và admin. Các luồng nghiệp vụ bao gồm quản lý hồ sơ thú cưng, đặt lịch, khám bệnh, kê đơn, grooming, lưu trú, hóa đơn, thanh toán, báo cáo và thông báo.

Phạm vi đánh giá: Tài liệu phản ánh code hiện tại trong thư mục Pet_Service, bao gồm backend đã được tái cấu trúc theo domain module nhưng chưa commit. Trọng tâm là ghi nhận các nền tảng thiết kế tốt mà nhóm đã xây dựng; các đề xuất còn lại được trình bày như bước hoàn thiện tiếp theo.

7.4.1. Thiết kế mô-đun của hệ thống

7.4.1.1. Kiến trúc tổng thể

Ở mức cao nhất, dự án tách frontend và backend thành hai ứng dụng độc lập. Frontend chịu trách nhiệm hiển thị, quản lý trạng thái giao diện và gọi HTTP API. Backend chịu trách nhiệm xác thực, phân quyền, kiểm tra nghiệp vụ và truy cập dữ liệu. PostgreSQL/Supabase nằm phía sau backend nên trình duyệt không truy vấn database trực tiếp.

```

React/Vite Frontend
  | HTTP + JSON + JWT
  v
Express Backend
  | Supabase server client / Prisma
  v
PostgreSQL Database
  
```

Luồng ví dụ - đặt lịch khám:

1. CustomerAppointmentsTab mở form và gọi customerAppointmentsApi.ts.
2. Request được gửi đến /api/customer/appointments kèm token.
3. authMiddleware xác thực người dùng và requireRole kiểm tra vai trò customer.
4. customer.routes.js chuyển dữ liệu sang customer-appointment.service.js.
5. Service kiểm tra thú cưng, thời gian, bác sĩ hoặc nhân viên rồi ghi dữ liệu qua Supabase.

Luồng trên cho thấy mỗi tầng có nhiệm vụ tương đối khác nhau. Đây là nền tảng của Separation of Concerns và giúp dự án có thể tiếp tục tách sâu hơn mà không phải viết lại toàn bộ hệ thống.

7.4.1.2. Modularity phía Backend

Đợt tái cấu trúc hiện tại đã chuyển backend từ cấu trúc routes/services rời rạc sang cấu trúc domain module tại backend/src/modules. Các module đang có gồm auth, users, customers, pets, services, appointments, staff, doctors, medical, grooming, boarding, billing, notifications và reports. Số lượng và mức hoàn thiện của các module được ghi theo source thực tế, không sử dụng con số 22 module như tài liệu tham khảo.

Nhóm	Module tiêu biểu	Trách nhiệm
Nền tảng	auth, users	Đăng nhập, đăng ký, khôi phục mật khẩu, tài khoản và cài đặt.
Khách hàng	customers, pets, appointments	Hồ sơ, thú cưng, lịch hẹn và lịch sử dịch vụ.
Chuyên môn	doctors, medical	Lịch bác sĩ, ca khám, bệnh án, đơn thuốc và chuyên khoa.
Vận hành	staff, grooming, boarding	Tiếp nhận, làm đẹp, lưu trú và chăm sóc hằng ngày.
Tài chính	billing, reports	Hóa đơn PDF, doanh thu và số liệu báo cáo.
Liên lạc	notifications	Thông báo trong hệ thống, email và lịch nhắc.

Các ranh giới kỹ thuật đã được tách rõ:

- app.js tạo Express app, cài middleware chung và mount các domain router.
- server.js chỉ mở cổng và bật reminder scheduler khi chạy trực tiếp.
- config chứa env và CORS; middlewares chứa xác thực/phân quyền.
- lib chứa JWT, Prisma và Supabase client; utils chứa helper HTTP.
- modules/index.js đóng vai trò nơi đăng ký prefix và router công khai.

```
for (const apiModule of apiModules) {
  app.use(apiModule.path, apiModule.router);
}
```

Cách mount module bằng danh sách giúp app.js không cần biết từng endpoint con. Khi thêm một domain router mới, nhóm chỉ cần đăng ký path và router trong modules/index.js. Đây là một ví dụ khá rõ của modularity và OCP ở cấp ứng dụng.

7.4.1.3. Modularity phía Frontend

Frontend được tổ chức theo hai hướng song song: chia theo vai trò ở thư mục features và chia theo loại thành phần ở components, services, types, utils. Cách tổ chức này phù hợp với React vì page có thể làm container, component chịu trách nhiệm hiển thị, service gọi API và type mô tả contract dữ liệu.

Khu vực	Ví dụ	Vai trò
features/auth	LoginPage, RegisterPage	Điều phối form và luồng xác thực.
features/customer	CustomerPortal, CustomerPetsPage	Màn hình và điều hướng của khách hàng.
features/doctor	DoctorPortal, DoctorExamScreen	Lịch khám, bệnh án, thống kê và khám bệnh.
features/staff	StaffPortal	Tiếp nhận, grooming, boarding và thanh toán.
features/admin	Dashboard, Users, Reports, Settings	Quản trị toàn hệ thống.
components/ui	Button, Dialog, Table, Input	Thành phần trình bày dùng chung.

Ví dụ về contract API phía customer:

```
export async function requestJson<T>(url: string, init?: RequestInit): Promise<T> {
  const response = await fetch(apiUrl(url), { ...init, headers: { ... } });
  const payload = await response.json().catch(() => ({}));
  if (!response.ok || payload?.ok === false) throw new Error(payload?.message);
  return payload as T;
}
```

requestJson.ts gom việc xác định URL, gắn header, parse JSON và chuyển lỗi HTTP thành Error. customerAppointmentsApi.ts và các service customer khác dùng helper này, vì vậy component không phải lặp lại phần kỹ thuật của fetch.

Kết quả đạt được và phần tiếp tục chuẩn hóa:

- Điểm tốt: feature theo role rõ, customer API và type được chia theo nghiệp vụ, component UI có khả năng tái sử dụng.
- Các role portal có không gian riêng nên việc phát triển song song và kiểm tra quyền truy cập thuận lợi hơn.
- Hệ thống type giúp nhiều luồng appointment, medication, notification và pet có contract rõ ràng.
- Phần tiếp tục chuẩn hóa là hợp nhất fetchWithAuth/requestJson và tách dần một số component dài.

7.4.2. Đánh giá Coupling và Cohesion

7.4.2.1. Coupling trong Pet Service

Coupling của hệ thống được đánh giá ở hai chiều: coupling giữa frontend với backend và coupling giữa các module trong backend. Kiến trúc hiện tại đã tránh Content Coupling ở cấp frontend vì trình duyệt không can thiệp trực tiếp vào dữ liệu nội bộ của backend. Hai phía giao tiếp qua HTTP, JSON và token.

Các điểm thể hiện coupling tương đối thấp:

- Frontend gọi service API thay vì gọi Supabase SDK hoặc biết câu truy vấn database.
- App mount module qua public contract path/router; chi tiết route nằm trong domain.
- Xác thực và phân quyền dùng authMiddleware/requireRole thay vì lặp ở từng endpoint.
- JWT, Prisma và Supabase client được tạo tập trung trong lib.
- Service frontend trả DTO/type thay vì chuyển nguyên row database vào component.
- Các route group có prefix riêng theo vai trò và nghiệp vụ, giúp thay đổi một nhóm API ít tác động sang nhóm khác.
- Compatibility re-export ở frontend giữ import cũ hoạt động trong quá trình tái cấu trúc.

Các điểm có thể giảm coupling thêm:

- Nhiều business service import Supabase client trực tiếp và gọi tên bảng/cột. Khi đổi data provider, nhiều file phải sửa.
- Một số response backend chứa class Tailwind như cls, dot hoặc mã màu. Backend vì vậy biết chi tiết hiển thị của frontend.
- Các cách gọi HTTP ở frontend chưa thống nhất, làm cơ chế token và error handling bị lặp.

Quan hệ	Mức hiện tại	Hướng tốt hơn
UI - API	Data/Stamp coupling qua DTO	Giữ DTO nhỏ, ổn định và có schema.
Route - Service	Khá lỏng ở auth/customer, chặt hơn ở medical	Controller/route mỏng, không query trực tiếp.
Service - Database	Coupling trực tiếp với Supabase	Repository contract hoặc data-access module.
Service - Email	Import và gọi trực tiếp	EmailSender/event publisher.

Kết luận Coupling: Phần lớn giao tiếp chính đã đi qua API, middleware, service và public module contract nên coupling ở cấp kiến trúc được kiểm soát khá tốt. Phần cần hoàn thiện tập trung ở data-access và việc chuẩn hóa HTTP client.

7.4.2.2. Cohesion trong Pet Service

Ở cấp thư mục, backend có cohesion khá tốt vì module được đặt theo domain. Ở cấp file, mức cohesion không đồng đều. Một số file có phạm vi rất rõ, nhưng một số file lớn đang gom nhiều use case và nhiều loại công việc khác nhau.

Ví dụ cohesion tốt:

- config/env.js chỉ đọc và chuẩn hóa cấu hình cổng, client URL và reminder interval.
- utils/http.js chỉ chứa helper sendError dùng để trả lỗi HTTP.
- app.js tập trung tạo ứng dụng Express; server.js tập trung khởi động process.
- customer-profile.service.js xử lý đọc/cập nhật hồ sơ customer.
- customer-medications.service.js tập trung lấy danh sách thuốc đang sử dụng.
- auth.routes.js và auth.service.js đã tách giao tiếp HTTP khỏi phần lớn logic xác thực.
- Các component doctor nhỏ như DoctorScheduleHeader, DoctorScheduleStats và VitalInput có trách nhiệm hiển thị rõ.

Các phạm vi nên tiếp tục tách nhỏ:

- medical.routes.js gần 2.000 dòng, vừa route, query, mapping, khám bệnh, đơn thuốc, chuyên khoa và notification.
- admin.service.js khoảng 1.700 dòng, chứa dashboard, user, dịch vụ, nhân sự, báo cáo, lịch bác sĩ và cài đặt.
- Một số component portal dài có thể tách thêm theo hook, view và modal để giữ cohesion ở cấp file.

Tệp	Cohesion ước lượng	Lý do
backend/src/app.js	Functional	Các phần cùng phục vụ việc tạo Express app.
backend/src/config/env.js	Functional	Chỉ xử lý cấu hình runtime.
medical.routes.js	Procedural/Communicational	Cùng luồng khám nhưng ôm nhiều trách nhiệm kỹ thuật.
admin.service.js	Logical	Nhiều chức năng admin được gom theo vai trò.
StaffBoardingTab.tsx	Logical/Procedural	Các thao tác boarding liên quan nhưng quá nhiều use case UI.

Hướng nâng cao cohesion:

- Tách theo use case nhỏ: exam, prescription, specialist order, notification.
- Tách mapper/formatter khỏi service truy vấn và khỏi route.
- Tách component lớn thành container, hook, view và modal chuyên trách.

Kết luận Cohesion: Cohesion ở cấp thư mục và nhiều file hạ tầng đã tốt sau tái cấu trúc. Một số file nghiệp vụ lớn là phần còn lại cần tách dần để Functional Cohesion được áp dụng đồng đều hơn.

7.4.3. Đánh giá các nguyên lý SOLID

7.4.3.1. Single Responsibility Principle (SRP)

SRP yêu cầu một thành phần chỉ nên có một nhóm lý do thay đổi. Trong Pet Service, ví dụ rõ nhất là `app.js` và `server.js`. Nếu thay cách mount route hoặc middleware chung thì sửa `app.js`; nếu thay cổng hoặc scheduler thì sửa `server.js`. Hai loại thay đổi không còn nằm trong cùng một file.

```
// app.js
const app = express();
app.use(cors(corsOptions));
for (const apiModule of apiModules) app.use(apiModule.path, apiModule.router);

// server.js
if (require.main === module) {
  app.listen(env.port);
  startReminderScheduler();
}
```

Các điểm tuân thủ SRP:

- `auth.middleware.js` chỉ xử lý bearer token, user context và role.
- `invoice-pdf.service.js` tập trung xây dựng hóa đơn PDF.
- Các component nhỏ như `DoctorScheduleHeader`, `DoctorScheduleStats` và `VitalInput` có phạm vi hiển thị rõ.
- Customer profile, medications và notifications có service chuyên biệt theo từng nhu cầu.

Phần cần hoàn thiện thêm:

- Một số file nghiệp vụ lớn vẫn có nhiều lý do thay đổi và nên được chia theo use case.

7.4.3.2. Open-Closed Principle (OCP)

OCP yêu cầu thành phần mở để mở rộng nhưng hạn chế sửa code lõi đã ổn định. `apiModules` là ví dụ tích cực: có thể thêm một route group bằng entry mới. Các component UI cũng có thể được kết hợp để tạo page mới mà không sửa implementation của Button, Dialog hoặc Table.

Các điểm đã hỗ trợ mở rộng:

- `apiModules` cho phép đăng ký thêm domain router mà `app.js` không cần biết endpoint con.
- Component UI dùng composition, giúp page mới tái sử dụng Button, Card, Dialog, Table và form control.
- Feature service và compatibility re-export giúp di chuyển module mà hạn chế làm hỏng caller cũ.

Phần có thể cải thiện:

- `App.tsx` phải sửa khi thêm role mới vì portal được chọn bằng chuỗi if.
- Một số bảng status/label có thể chuyển sang registry hoặc policy để mở rộng thuận tiện hơn.

7.4.3.3. Liskov Substitution Principle (LSP)

Pet Service chủ yếu dùng JavaScript/TypeScript theo phong cách module và composition, không có nhiều class cha - class con trong nghiệp vụ. Vì vậy, không nên cố chứng minh LSP bằng một hệ kế thừa không tồn tại. Cách phù hợp hơn là xem các implementation cùng contract có thay thế được cho nhau hay không.

Nền tảng hiện tại:

- Frontend có TypeScript interface mô tả DTO của appointment, medication, notification và pet.
- Các service thường trả dữ liệu theo shape ổn định và lỗi dưới dạng Error/message.
- Các module index tạo public contract ở cấp module và giúp caller không cần import mọi file nội bộ.
- Cách chuẩn hóa DTO hiện có là tiền đề để bổ sung repository contract trong bước tiếp theo.

Điều kiện để đáp ứng LSP tốt hơn:

- Định nghĩa contract cho repository: kiểu dữ liệu, điều kiện lỗi và semantics của từng phương thức.
- Tạo SupabaseRepository và FakeRepository cùng tuân thủ contract.
- Viết contract test dùng chung để đảm bảo implementation mới không làm thay đổi hành vi của service.

7.4.3.4. Interface Segregation Principle (ISP)

ISP yêu cầu caller chỉ phụ thuộc vào nhóm chức năng mà nó thật sự sử dụng. Dự án đã thể hiện nguyên lý này khá rõ ở customer API, component props và các domain router. Một số service lớn có thể tiếp tục được chia nhỏ khi quy mô tăng.

Nền tảng hiện tại	Điểm tốt	Hướng mở rộng
Customer service modules	Appointment, boarding, medication, notification, pet và profile đã tách riêng.	Giữ contract nhỏ và bổ sung schema dùng chung.
Doctor/staff feature service	Đã gom theo đúng portal và che giấu HTTP khỏi phần lớn component.	Tách tiếp ExamApi, BoardingApi, PaymentApi khi số use case tăng.
Backend domain modules	Auth, customer, doctor, staff và admin có public router/service rõ.	Tách admin report/settings/catalog thành service chuyên biệt.

Khi tách interface/service theo use case, component hoặc route chỉ import phần cần dùng. Điều này làm giảm phụ thuộc dư thừa, dễ mock khi test và giúp quyền truy cập nghiệp vụ rõ ràng hơn.

7.4.3.5. Dependency Inversion Principle (DIP)

DIP yêu cầu logic cấp cao phụ thuộc abstraction thay vì chi tiết hạ tầng. Pet Service đã có service layer giữa UI và backend, đồng thời có lib cho Supabase/Prisma/JWT. Tuy nhiên đây mới là bước đầu vì business service vẫn import chi tiết hạ tầng trực tiếp.

Phụ thuộc hiện tại:

```
Route -> Service -> Supabase SDK -> PostgreSQL
                        |
                        +--> Email service -> Nodemailer
                        +--> Storage -> Supabase Storage
```

Thiết kế hướng DIP đề xuất:

```
Use case service -> AppointmentRepository interface
                  -> EmailSender interface
                  -> FileStorage interface

Composition root -> SupabaseAppointmentRepository
                  -> NodemailerEmailSender
                  -> SupabaseFileStorage
```

Với cách thứ hai, service không cần biết dữ liệu đến từ Supabase hay Prisma, email được gửi qua Gmail hay provider khác. Khi unit test, nhóm truyền fake repository và fake email sender để kiểm tra nghiệp vụ mà không kết nối dịch vụ thật.

7.5. Đánh giá theo các tính chất mô-đun lý tưởng

Tính chất	Đánh giá	Bằng chứng
Decomposable	Khá	Frontend/backend tách riêng; backend theo domain; frontend theo role/feature.
Composable	Khá	Page ghép component; app ghép router qua apiModules; service phối hợp theo luồng.
Understandable	Khá	Cấu trúc thư mục rõ nhưng file lớn làm tăng thời gian đọc.
Continuity	Khá - đang hoàn thiện	Phần lớn thay đổi nằm trong feature/domain; DTO và data-access có thể ổn định thêm.
Isolation	Khá - đang hoàn thiện	Middleware/lib/service đã cô lập nhiều chi tiết; event/repository sẽ tăng mức độc lập.

Đánh giá chung: Pet Service đáp ứng tốt phần lớn tiêu chí mô-đun: Decomposable, Composable và Understandable đều ở mức khá. Continuity và Isolation đã có nền tảng, cần được củng cố thêm bằng repository, schema/DTO ổn định và tách side effect.

7.6. Các điểm cần tiếp tục hoàn thiện

Qua quá trình tái cấu trúc, Pet Service đã hình thành nền tảng khá rõ về module, service layer, middleware và role-based frontend. Để giữ chất lượng này khi hệ thống tiếp tục mở rộng, nhóm xác định một số hạng mục nên hoàn thiện thêm. Các hạng mục dưới đây chiếm phần nhỏ hơn so với những phần thiết kế đã làm tốt.

7.6.1. Duy trì quy mô file hợp lý

Cấu trúc domain/feature đã giúp khoanh vùng tốt phần lớn chức năng. Bước tiếp theo là tách một số file còn dài theo use case để giảm merge conflict và giúp unit test tập trung hơn.

7.6.2. Tiếp tục làm mỏng Route

Nhiều route auth/customer đã gọi service khá rõ ràng. Nhóm có thể áp dụng đồng đều cách này cho medical, doctor, staff và services để route chỉ nhận request, gọi use case rồi trả response.

7.6.3. củng cố ranh giới Data-access

Supabase client hiện đã được khởi tạo tập trung trong lib. Việc bổ sung repository theo domain sẽ giúp giữ tên bảng, quan hệ join và cấu trúc row ở một lớp thấp hơn, qua đó bảo vệ business service khi schema thay đổi.

7.6.4. Thống nhất Contract API

Customer API đã có requestJson và TypeScript type làm nền tảng tốt. Có thể mở rộng cùng chuẩn này sang doctor, staff và admin; response domain nên tách khỏi màu sắc hoặc Tailwind class của giao diện web.

7.6.5. Tăng khả năng cô lập Side effect

Email, PDF và storage đã có service riêng thay vì nằm hoàn toàn trong route. Bước hoàn thiện là đặt interface/event ở trước các service này để retry, test và thay provider thuận tiện hơn.

Hạng mục hoàn thiện	Nền tảng đã có	Bước tiếp theo
Tách file lớn	Đã có ranh giới domain và feature.	Tách tiếp theo use case trước khi thêm tính năng lớn.
Data-access	Supabase client được tạo tập trung trong lib.	Bổ sung repository ở domain trọng yếu.
API contract	Customer API đã có requestJson và TypeScript DTO.	Chuẩn hóa ApiResult/ApiError cho toàn frontend.
Presentation mapping	Backend đã trả status/label nhất quán ở nhiều luồng.	Chuyển màu/class cụ thể về frontend.

7.7. Lộ trình cải thiện thiết kế

Lộ trình dưới đây không yêu cầu viết lại dự án. Mục tiêu là cải thiện dần theo rủi ro, giữ nguyên chức năng đang chạy và tạo ranh giới tốt hơn cho phần phát triển tiếp theo.

7.7.1. Giai đoạn ngắn hạn

6. Thống nhất một ApiClient cho frontend: apiUrl, auth header, JSON, timeout, ApiError và invalid-session.
7. Tách medical.routes.js thành route mỏng và service theo exam, prescription, specialist order, notification.
8. Tách component trên 700-800 dòng thành container, hook, view và modal.
9. Chuẩn hóa response theo { ok, data, message } hoặc contract tương đương.
10. Chuyển Tailwind class và màu trạng thái khỏi backend về frontend.

7.7.2. Giai đoạn trung hạn

11. Tách admin.service.js và staff.service.js theo domain/use case.
12. Tạo AppointmentRepository, PetRepository, BoardingRepository và InvoiceRepository.
13. Bổ sung validation schema tại API boundary cho input quan trọng.
14. Tạo global error middleware và error type chung cho backend.
15. Chuyển ownership thật cho reports, grooming và billing thay vì chỉ re-export.

7.7.3. Giai đoạn dài hạn

16. Áp dụng dependency injection tại composition root cho repository, email, storage và clock.
17. Dùng domain/application event cho AppointmentConfirmed, ExamCompleted, InvoicePaid và BoardingUpdated.
18. Dùng OpenAPI hoặc shared schema để đồng bộ contract frontend-backend.
19. Viết contract test cho repository và integration test cho các luồng nghiệp vụ chính.

7.7.4. Tiêu chí hoàn thành

Tiêu chí	Kết quả mong đợi
Route mỏng	Không truy vấn database trực tiếp; chỉ validate, gọi use case và trả response.
Service tập trung	Mỗi service có một nhóm use case liên quan và quy mô đọc được.
Data access thay thế được	Business logic chạy với repository thật hoặc fake.
Frontend API thống nhất	Mọi feature dùng cùng auth/error/JSON behavior.
Side effect cô lập	Email/storage lỗi có cơ chế retry hoặc xử lý riêng.

7.8. Kết luận

Dựa trên các tiêu chí của tài liệu Good Design, Pet Service đã có nền tảng thiết kế tương đối tốt. Việc tách frontend và backend, chia frontend theo role/feature, chia backend theo domain module, tách app khỏi server và đưa config, middleware, lib vào khu vực riêng giúp cấu trúc dự án dễ hiểu hơn so với cách tổ chức routes/services phẳng trước đây.

Hệ thống thể hiện khá rõ nhiều nguyên lý quan trọng: SRP qua ranh giới app/server và các service nhỏ; OCP qua apiModules và khả năng ghép component; ISP qua customer API chuyên biệt; DIP ở mức nền tảng qua service layer, lib và middleware dùng chung. TypeScript DTO cùng public module contract cũng tạo tiền đề tốt để tiến tới LSP theo hướng implementation có thể thay thế.

Nhóm 23 đánh giá khoảng 70% tiêu chí Good Design đã được triển khai tốt và có bằng chứng trực tiếp trong source. Khoảng 30% còn lại là công việc hoàn thiện, không phải yêu cầu viết lại hệ thống: tách một số file lớn, bổ sung repository, chuẩn hóa DTO/HTTP client và cô lập side effect. Khi các bước này được thực hiện, dự án sẽ dễ bảo trì, kiểm thử và mở rộng hơn nữa.

Kết quả tổng hợp: Khoảng 70% tiêu chí Good Design đã được thể hiện tốt trong code hiện tại: modularity rõ, service/middleware dùng chung, frontend theo role-feature và backend theo domain. Khoảng 30% còn lại là các bước hoàn thiện về file lớn, repository, contract API và side effect.

Tài liệu và mã nguồn đối chiếu

- Tài liệu Thiết kế tốt.pdf - tài liệu tham khảo về Modularity, Coupling, Cohesion và SOLID.
- README.md, docs/architecture.mdx, docs/frontend.mdx, docs/backend.mdx.
- backend/src/app.js, server.js, config/, middlewares/, lib/, modules/ và utils/.
- frontend/src/app, components, features, services, types và utils.

Thành viên thực hiện

STT	Họ và tên	Nhóm
1	Trần Đức Nam Anh	23
2	Nguyễn Đức Hiếu	23
3	Triệu Trường Giang	23